

TRACE AI

Product Requirements Document

AI-powered log analysis and incident investigation platform

Version	1.0
Product area	AI/ML · Developer tools · Jev API
Document purpose	Implementation brief for an AI coding platform

Purpose: turn raw application logs into structured, evidence-based incident reports using machine learning and the Jev AI API.

1. Product overview

Product: TRACE AI

Type: AI-powered log analysis and incident investigation platform.

Vision: Help developers and IT teams detect unusual system behaviour, investigate errors and identify potential root causes without manually examining thousands of log entries.

Core objective: Turn raw application logs into structured, evidence-based incident reports using machine learning and the Jev AI API. The product must be functional, not a static dashboard or chatbot.

2. Problem statement

Application failures generate large volumes of logs. Important errors can be buried among routine events; multiple errors may share an underlying cause; and reconstructing the sequence leading to a failure can be time-consuming. TRACE AI reduces manual investigation effort through anomaly detection, error correlation and evidence-based incident summaries.

3. Target users

User	Primary need
Backend developers	Find the source of application failures.
DevOps engineers	Investigate abnormal system behaviour.
Site reliability engineers	Identify and investigate incidents.
Software engineering students	Learn log analysis and anomaly detection.
Small development teams	Investigate failures without expensive monitoring tools.

4. Core features

4.1 Log ingestion

Upload .log, .txt and .csv files. Parse timestamps, severity, messages and available service names. Detect malformed or missing fields; show parsed/rejected counts; provide sample logs.

4.2 Anomaly detection

Use Python and scikit-learn, initially Isolation Forest. Extract features such as error frequency, severity and time-based counts. Flag anomalous events with scores and expose supporting features. Distinguish anomalies from confirmed incidents.

4.3 Jev AI incident analysis

Use the official Jev API documentation; do not invent endpoints or schemas. Classify incidents (database, networking, application, resource exhaustion), assess severity, identify related patterns and return structured evidence-based explanations. Flag uncertain diagnoses for manual review.

4.4 Incident correlation

Group potentially related errors using timestamps, service names, patterns and correlation rules. Show event sequence and supporting evidence. Temporal order alone must not be treated as proof of causation.

4.5 Incident dashboard

Show total entries analysed, anomalies, incidents, severity distribution, error frequency over time and recent incidents. All metrics and charts must derive from uploaded data, never hardcoded values.

4.6 Incident reports

Each incident has an ID, detection time, severity, category, related logs, anomalies, possible causes, evidence, recommended investigation steps and uncertainty. Export reports as PDF or JSON.

5. User interface and design

Create a professional developer tool, not a generic AI chat interface. Use a charcoal dark theme, subtle borders, emerald green for healthy states, amber for warnings and red for critical incidents. Use Inter or a similar readable font, clear typography, compact cards and responsive layouts. Avoid excessive gradients, decorative elements and unnecessary animation.

Page	Purpose
Overview	Dashboard and system statistics.
Upload Logs	Upload and validate files.
Analysis	View anomaly detection and processing results.
Incidents	Browse, search and filter incidents.
Incident Details	Investigate a specific incident.
Settings	Configure API key and analysis preferences.

6. Technology stack

Layer	Technology	Purpose
Frontend	Next.js + TypeScript	Responsive interface
Styling	Tailwind CSS	Consistent UI
Charts	Recharts	Visualize trends
Backend	Python + FastAPI	Log processing and API
ML	scikit-learn	Anomaly detection
AI	Jev API	Structured incident classification
Database	SQLite + SQLAlchemy	Persist results
Validation	Pydantic	Validate requests/responses
PDF export	ReportLab	Incident reports

7. System architecture

Flow: Next.js frontend → FastAPI backend → log parsing and feature extraction → ML anomaly detection → incident correlation → Jev API analysis → SQLite storage → dashboard and report export.

The backend coordinates all components. The frontend must never call Jev directly with a secret key.

8. Jev API integration

- Use the official Jev API documentation for the exact endpoint, request and response formats.
- Send structured incident evidence, not entire log files.
- Validate and parse API responses before displaying or storing them.
- Handle timeouts, rate limits, invalid responses and service failures.

- Keep the API key in a server-side environment variable named JEV_API_KEY; never commit credentials.
- If evidence is insufficient, display “Insufficient evidence” rather than fabricate a diagnosis.

9. Data model

Entity	Important fields
LogEntry	ID, timestamp, severity, service, message
Anomaly	ID, log ID, anomaly score, detection time
Incident	ID, title, severity, category, status, detected time
IncidentEvidence	Incident ID, log ID, evidence type
Diagnosis	Incident ID, possible cause, confidence, explanation
AnalysisRun	ID, filename, status, start time, completion time

10. User workflow

- Upload a supported log file and review validation results.
- Parse logs, extract features and run anomaly detection.
- Group potentially related anomalies and errors into candidate incidents.
- Send relevant, sanitized incident evidence to Jev and process the response.
- Review the dashboard and inspect incident evidence.
- Export an incident report as PDF or JSON.

11. Non-functional requirements

- Performance: target under 30 seconds for local parsing and anomaly detection of 10,000 entries on a typical development machine, excluding external API latency.
- Reliability: failed Jev requests must not erase completed log analysis.
- Security: protect credentials, validate uploads and enforce file-size limits.
- Privacy: send only relevant, sanitized evidence to the external API.
- Usability: the main workflow should require no configuration beyond providing the API key.
- Maintainability: separate the parser, ML model, correlation logic and AI integration into independent modules.

12. Testing and evaluation

Include labelled sample logs for database connection exhaustion, repeated HTTP 500 errors, high memory usage, network timeouts and normal activity.

Metric	Purpose
Precision	How many flagged anomalies are genuine.
Recall	How many known anomalies were detected.
F1-score	Balance between precision and recall.
False-positive rate	How often normal events are incorrectly flagged.
Diagnosis accuracy	How often a suggested cause matches the labelled scenario.

Calculate all evaluation scores from actual test results. Evaluate anomaly detection separately from Jev incident classification. Do not hardcode or claim unverified scores.

13. Out of scope for V1

- Live server monitoring
- Automatic remediation
- Kubernetes integration
- Slack or email alerts
- Multi-tenant enterprise accounts
- Complex distributed tracing
- Automatic production deployments

14. Acceptance criteria

- Users can upload and analyse supported log files.
- Anomalies are detected using an actual ML model.
- Related log entries can be grouped into candidate incidents.
- Jev integration uses a server-side API key.
- Users can inspect evidence for each diagnosis.
- Dashboard metrics reflect actual analysis results.
- Incident reports can be exported.
- API failures and malformed logs are handled gracefully.
- Sample data and setup instructions are included.
- The application runs locally using documented commands.

15. Instructions for the AI coding platform

Build TRACE AI according to this PRD. Create a fully functional, responsive web application, not a static mockup. Prioritize the complete workflow from log upload through anomaly detection, incident correlation, Jev analysis and report export. Use the specified stack and keep the architecture modular.

Implement the real Jev integration only after checking its official documentation. If credentials are unavailable, provide a clearly labelled demo mode using deterministic sample responses, never fake a successful live API call. Include loading, empty, error and success states; a README with setup instructions; a `.env.example` with placeholders; and tests for parsing, anomaly detection, correlation and API error handling.

Do not claim that a suggested root cause is confirmed unless validated. Show evidence and uncertainty clearly. Never expose API keys or send raw sensitive logs unnecessarily.